

Overall Merit: Accept

Summary This paper addresses a deceptively hard problem: how to build hardware performance counters that produce accurate CPI (Cycles Per Instruction) stacks on superscalar out-of-order processors. CPI stacks decompose total execution time into a base component plus penalties from cache misses, branch mispredictions, TLB misses, and other events. They are widely used by developers and architects to identify bottlenecks, but existing approaches (the naive method, the naive_non_spec variant, and the IBM POWER5 mechanism) all produce significant errors because they reason bottom-up from event counts without accounting for how miss events actually interact with out-of-order execution.

The main contribution is a top-down approach grounded in *interval analysis*, a performance model that partitions execution into intervals defined by disruptive miss events. Interval analysis reveals the true per-event penalty by reasoning about how each miss type disrupts instruction flow: frontend misses (I-cache, I-TLB, branch mispredictions) cause pipeline drain and refill, while long backend misses (L2 D-cache, D-TLB) block the ROB head. The model also captures overlaps, e.g., multiple long D-cache misses within one ROB window share latency, and frontend misses overlapping with backend stalls should not be double-counted.

From this analysis, the authors propose two hardware structures: the FMT (frontend miss event table) and its simplified variant sFMT, which track frontend miss penalties per branch and avoid counting misses along mispredicted paths. Combined with straightforward backend and resource stall counters, the resulting CPI stacks achieve average errors of 2.5% (FMT) and 2.7% (sFMT) across SPECint2000, with all maximum component errors below 4%. This is far better than the naive approach (>30% errors) and the POWER5 mechanism.

Strengths

1. The interval analysis framework does real explanatory work. Rather than just proposing a counter design, the paper derives the design from a performance model. Figures 2 through 8 walk through each miss type's interval behavior, making it clear *why* existing approaches fail and *what* the counters need to measure.
2. The paper is thorough in handling interactions between miss events (Section 3.3). The overlap between frontend and backend misses, and the treatment of multiple long D-cache misses within one ROB window (Figure 8), are the hardest parts to get right and are where existing approaches break down most severely.
3. The sFMT simplification is practically important: it achieves nearly the same accuracy as the full FMT with significantly less storage (no per-branch I-cache/I-TLB counters), making adoption more realistic.

Weaknesses

1. The evaluation uses only SPECint2000 on a single SimpleScalar/Alpha configuration. While the interval model is general, the accuracy claims depend on the assumption that the processor is “well-balanced” (ROB large enough, dispatch width matching pipeline). It would strengthen the paper to show sensitivity to configurations where this assumption weakens.
2. The paper acknowledges but does not deeply explore SMT. On SMT processors, resource contention between threads makes per-thread CPI attribution much harder, and the interval model’s single-thread framing would need serious reworking.

Detailed Comments I found the comparison between the four CPI stack methods (naive, naive_non_spec, POWER5, FMT/sFMT) in Figure 11 to be the most persuasive part of the paper. The gcc and gap benchmarks are especially striking: the naive approach produces negative base CPI components (clearly meaningless), the POWER5 approach improves but still shows substantial errors, while FMT/sFMT closely track the simulation-based ground truth. This visual evidence makes the accuracy claims concrete.

A subtle but important point is the paper’s “well-balanced processor” assumption (Section 3). The claim that achieved IPC approximates dispatch width under ideal conditions is what allows the base CPI to be computed as a residual. This is reasonable for practical processor widths (2 to 8-way), but could break for very wide or very narrow machines. The paper is honest about this, though it does limit how far you can generalize the results.

Discussion Questions

1. The interval model assumes miss events are the primary disruptors of steady-state execution. How well does this hold for workloads with heavy data dependencies or long-latency functional unit chains where the ROB fills even without cache misses?
2. Modern processors have much deeper pipelines, larger ROB, and more complex memory hierarchies than the SimpleScalar model used here. Which parts of the interval analysis would need revision for a 2020s-era core with 300+ ROB entries and multi-level TLBs?
3. Could the interval analysis framework be extended to attribute energy rather than cycles? If each interval type has a characteristic power profile, could we build “energy stacks” with similar hardware?